

Practica 1

Ejercicio 1

Ejercicio 2

Realice una solución concurrente de grano grueso (utilizando `<>` y/o `<await B; S>`) para el siguiente problema. Dado un número N verifique cuántas veces aparece ese número en un arreglo de longitud M. Escriba las pre-condiciones que considere necesarias.

```
// Precondiciones: el tamaño del arreglo y la cantidad de procesos es multi-
// plo de 2
int n; int total = 0;
int porcion = m/p; int m; int p;
int v[1,m];

process buscar()[id = 0 to p-1]:
    int cant = 0;
    for (i = id*porcion; i < (id*porcion) + porcion; i++) {
        if (v[i] = n)
            cant++;
    }
    <total += cant>
}
```

Ejercicio 3

Ejercicio 4

```

queue recursos [5];
recurso r;

process usarRecursos(id = 1.. n)::
{ while (true)
  {
    <await not empty(recursos); pop(r, recursos)>
    // usa recurso
    <push(r, recursos)>
  }
}

```

Ejercicio 5

a_

```

doc documento;
int mutex = 1

process persona(id = 1..n)::
{ <Imprimir(documento);> }

```

b_

```

doc documento;
queue fifo [n];
int siguiente = -1;

process persona(id = 0..n-1)::
{
  <if (siguiente = -1); siguiente = id
  else push(id, fifo)>;
  <await (siguiente == id)>;
  Imprimir(documento);
  <if (empty(fifo)) siguiente = -1
  else siguiente = pop(fifo)>;
}

```

c_

```
// Precondiciones: asumo que la colaOrdenada ya tiene
// implementada su funcionalidad, donde se agregan los id
// a la cola por orden segun el id.

doc documento;
queue colaOrdenada [n];
int siguiente = -1;

process persona(id = 0..n-1)::
{
  <if (siguiente = -1); siguiente = id
  else push(id, fifo)>;
  <await (siguiente == id)>;
  Imprimir(documento);
  <if (empty(fifo)) siguiente = -1
  else siguiente = pop(fifo)>;
}
```

d_

```
// Precondiciones: asumo que la colaOrdenada ya tiene
// implementada su funcionalidad, donde se agregan los id
// a la cola por orden segun el id.
colaOrdenada espera;
int actual = -1;
bool listo = false , ok = false;

process persona(id = 0..n-1)::
{ <agregar(espera, id)>;
  <await (actual == id)>;
  ok = true;
  // Usa la impresora
  <await(listo)>
  listo = false;
}
process coordinador::
```

```

{
  for i = 0..n-1
    {<await(not empty(espera)); sacar(espera, actual)>
     <await (ok)>; ok = false;
     listo = true;
     <await (not listo)>
    }
}

```

Ejercicio 6

6. Dada la siguiente solución para el Problema de la Sección Crítica entre dos procesos (suponiendo que tanto SC como SNC son segmentos de código finitos, es decir que terminan en algún momento), indicar si cumple con las 4 condiciones requeridas:

int turno = 1;	
Process SC1:: { while (true) { while (turno == 2) skip; SC; turno = 2; SNC; } }	Process SC2:: { while (true) { while (turno == 1) skip; SC; turno = 1; SNC; } }

No cumple con exclusion mutua, ya que modifica el valor de la variable compartida turno, y no lo hace de forma atomica, eso